Redes Neuronales biologicamente inspiradas aplicadas al aprendizaje por refuerzo profundo

Adolfo Banchio

February 22, 2026

Director: Dr. Martín A. Domínguez
Facultad de Matemática, Astronomía, Física y Computación (FaMAF)



Estructura de la presentación

Motivación

Marco Teorico

Agente Bioinspirado e implementacion

Definición del Problema

Algoritmo de Entrenamiento: TD3

Dificultades y Estrategias de Mejora

Resultados

Conclusiones

Motivación

Marco Teorico

Agente Bioinspirado e implementacion

Definición del Problema

Algoritmo de Entrenamiento: TD3

Dificultades y Estrategias de Mejora

Resultados

Conclusiones

Contexto

- El Aprendizaje por Refuerzo Profundo ha mostrado resultados sobresalientes y prometedores en areas como:
  - Tareas de control: robotica
  - Juegos Complejos: Atari
  - Entrenamiento de LLMs: fine tunning de chatGPT.
- Predomina el uso de arquitecturas neuronales tradicionales:
  - MLP
  - CNN
  - RNN

¿Que ocurre si utilizamos arquitecturas inspiradas biologicamente?

Antecedentes

Trabajos anteriores buscaron responder dicha pregunta y lograron:

- Integrar una **arquitectura neuronal bioinspirada** (Tap Withdrawal Circuit de *C. elegans*) en el marco del aprendizaje por refuerzo.
- Plantear nuevos modelos de dinamica neuronal (**modelo Fiuri**) y compararlos con modelos estándar (IandF, Izikievich).
- Utilizar metodos de entrenamiento alternativos al descenso por el gradiente (Optimizacion bayesiana, algoritmos geneticos y busquedas aleatoreas).

Nos preguntamos si es posible:

- Implementar de manera eficiente y vectorizada el agente
- Permitir el uso del el descenso por el gradiente
- Integrar a los algoritmos de Deep RL estandar

Objetivo y Desafios

Objetivo

Integrar la arquitectura **TWC + FIU** al framework de PyTorch e intentar resolver un problema de control continuo utilizando un algoritmo estandar Deep RL.

Desafios

- Las arquitecturas bioinspiradas presentan:
  - Dinámica interna recurrente
  - No linealidades inspiradas biológicamente
  - Posibles problemas de diferenciabilidad
- La mayoría de los trabajos previos:
  - Evitan el descenso por gradiente
  - Utilizan métodos evolutivos o búsqueda directa en el espacio de políticas

Motivación

Marco Teorico

Agente Bioinspirado e implementacion

Definición del Problema

Algoritmo de Entrenamiento: TD3

Dificultades y Estrategias de Mejora

Resultados

Conclusiones

Inteligencia Artificial y ML

- Busqueda de **agentes** capaces de tomar la decision correcta en funcion de la información disponible.

Aprendizaje automático (Machine Learning)

Un programa de computadora aprende de la experiencia E con repecto a alguna clase de tareas T y una medida de desempeño P, si su desempeño en tareas de T, medidio por P, mejora con la experiencia E.

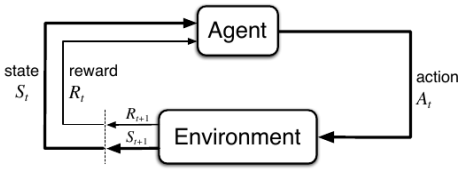
Segun el tipo de experiencia y medida de desempeño, se divide en:

- Aprendizaje supervisado
- Aprendizaje no supervisado
- **Aprendizaje por refuerzo**

Aprendizaje por Refuerzo (RL)

Un agente interactúa con un entorno dinámico donde:

- Observa estados s_t
- Ejecuta acciones a_t
- Recibe recompensas r_{t+1}



Con el objetivo de maximizar la recompensa final acumulada:

$$\max_{\pi} \ \mathbb{E}[R]$$

Principales desafíos:

- Problema de asignación de crédito
- Exploración vs explotación
- Retroalimentación escasa y diferida

Procesos de Decisión de Markov (MDP)

La interacción **Agente–Entorno** se formaliza mediante un Proceso de Decisión de Markov (MDP).

Definición

Un MDP se define como la tupla $\mathcal{M} = \langle S, A, P, R, \gamma \rangle$ donde:

- S: conjunto de estados posibles.
- A: conjunto de acciones disponibles.
- $P(s' \mid s, a)$: probabilidad de transición al estado s' al ejecutar a en s.
- $R(s, a, s')$: recompensa inmediata esperada asociada a la transición.
- $\gamma \in [0, 1]$: factor de descuento.

Propiedad de Markov

El estado actual s_t contiene toda la información necesaria para predecir la evolución futura del entorno:

$$\mathbb{P}(s_{t+1}, r_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0) = \mathbb{P}(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

Componentes del Aprendizaje por Refuerzo

El Agente toma acciones siguiendo una **Política**: mapeo de estados y observaciones del entorno a una accion que debera tomar el agente.
Objetivo del agente: encontrar la politica que maximice el retorno final.

Retorno y Politica Optima

Dado una secuencia de recompensas $r_{t+1}, r_{t+2} \cdots$, se define el retorno como:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$$

La politica optima sera $\pi^* = \arg\max_\pi J(\pi)$ donde $J(\pi) = \mathbb{E}[G_0]$

Componentes del Aprendizaje por Refuerzo

Ecuaciones de Bellman

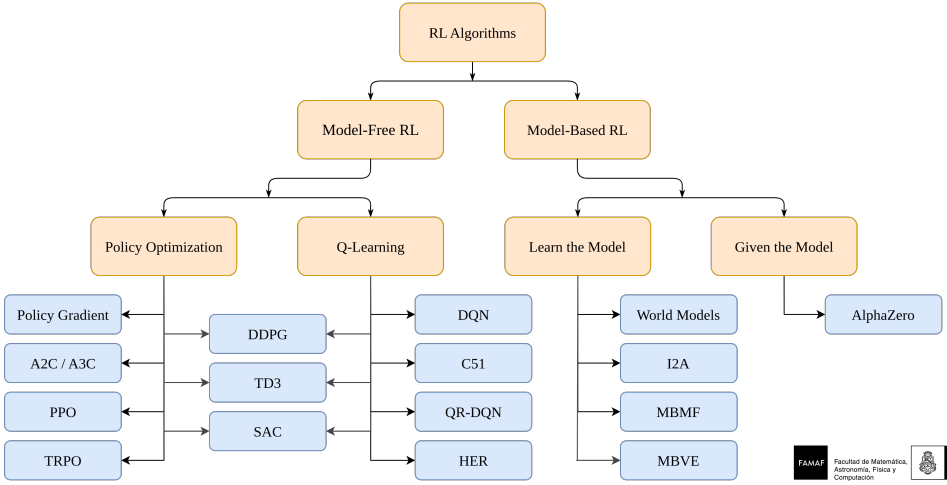
Ecuaciones recursivas que nos permiten estimar la recompensa esperada segun la situacion actual del Agente en el entorno.

- **Función de valor de estado**: estima la recompensa acumulada esperada al partir del estado s y seguir la política π.
  - $V^{\pi}(s) = \mathbb{E}_{\pi}\left[r_{t+1} + \gamma V^{\pi}(S_{t+1}) \mid S_t = s\right]$
- **Función de valor de acción** estima la recompensa acumulada esperada tras ejecutar la acción a en el estado s y continuar siguiendo π.
  - $Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[r_{t+1} + \gamma Q^{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a\right]$.

En espacios de accion continuos:

- Computar $\arg\max_a Q(s,a)$ no es factible
- Se aproximan dichas funciones mediante redes neuronales

Algoritmos de RL



Métodos Actor-Crítico

Se utilizan dos redes neuronales:

- **Actor**: aproxima la política
- **Critic**: aproxima la función de valor

Aproximaciones paramétricas:

$$Q^{\pi}(s, a) \approx Q(s, a; \theta)$$

$$\pi(a \mid s) \approx \pi(a \mid s; \phi)$$

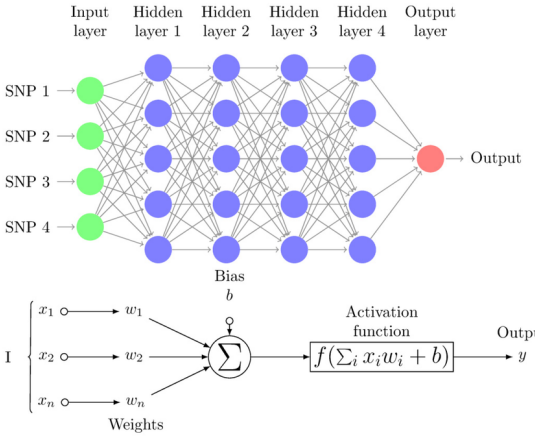
- El Actor produce directamente la acción
- El Critic evalúa dicha acción

Redes Neuronales

- Comunmente utilizadas como aproximadores de funciones universales.
- Se entrenan mediante metodos basados en el **descenso por el gradiente**.

Limitaciones:

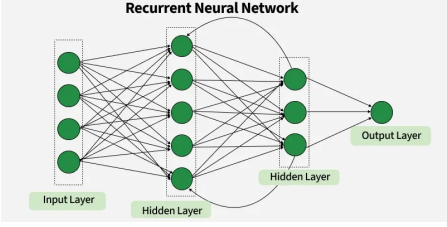
- Arquitectura feed-forward
- No poseen memoria interna
- No acomulan informacion en una secuencia temporal



Redes Recurrentes y Dependencia Temporal

Cuando la decision depende del historial, usamos **Redes Recurrentes**:

- Introduce la nocion de estado interno
- Le da valor a la observacion de secuencias temporales
- Se entrenan mediante **BPTT** y descenso por el gradiente.



Idea General

Aprovechar los algoritmos de entrenamiento de DeepRL de RNNs para mi futuro modelo bioinspirado

Descenso por el Gradiente y BPTT

- El entrenamiento se basa en minimizar una **función de error** $L(\theta)$.
- El **descenso por el gradiente** ajusta los parámetros en la dirección que reduce dicho error:

$$\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$$

Redes Recurrentes

- La salida actual depende del estado interno previo.
- El error final depende de decisiones pasadas.
- Se utiliza **Backpropagation Through Time (BPTT)**:
  - Se despliega la red en el tiempo.
  - El gradiente se propaga hacia atrás a lo largo de la secuencia.

Inspiracion biologica

Dinamica neuronal Fiuri (**FIU**):

- Diseño simplificado
- Discreto en el tiempo
- Cada neurona tiene:
  - Dos estados: *interno* (E) y *externo* (O)
  - Dos parametros: *umbral de disparo* (Th) y *factor de decaimiento* (d).
- Tipos de sinapsis:
  - Exitatorias
  - Inhibitorias
  - Uniones electricas (Gap Junctions)

Conectoma del *Tap Withdrawal Circuit* del nematodo **C Elegans**.

- Encargado de la respuesta reflejo a estimulos externos
- Compuesto por 11 neuronas: Sensoriales, internas y motoras.
- Presenta conexiones recurrentes esparsas.

Motivación

Marco Teorico

Agente Bioinspirado e implementacion

Definición del Problema

Algoritmo de Entrenamiento: TD3

Dificultades y Estrategias de Mejora

Resultados

Conclusiones

Dinamica Neuronal FIU

$$S_n = E_n + \sum_{j=1}^{m} I_{in}^{j}$$

$$O_n = \begin{cases} S_n - T_n, & \text{si } S_n > T_n \\ 0, & \text{en otro caso} \end{cases}$$

$$E_n = \begin{cases} S_n - T_n, & \text{si } S_n > T_n \\ E_n - d_n, & \text{si } S_n \leq T_n \ \wedge \ S_n = E_n \\ S_n, & \text{en otro caso} \end{cases}$$

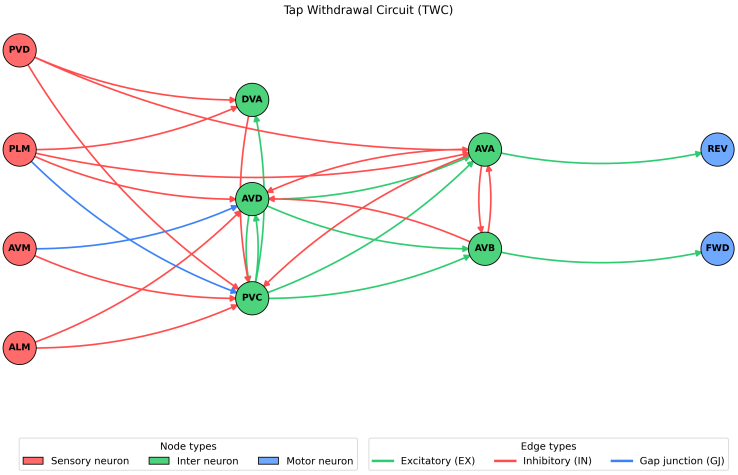
$$I_{in}^{j} = \begin{cases} \omega_j O_j, & \text{gap junc. y } O_j \geq E_n \\ -\omega_j O_j, & \text{gap junc. y } O_j < E_n \\ \omega_j O_j, & \text{exc. química} \\ -\omega_j O_j, & \text{inh. química} \end{cases}$$

- Variables de estado
  - E_n: estado interno
  - O_n: estado externo
- Parametos de la dinamica
  - T_n: umbral de disparo
  - d_n: factor de decaimiento

Desafio

Implementar esta dinamica para poder aplicar el descenso por el gradiente y BPTT.

La arquitectura TWC



Implementación del Agente: PyUriTwc

- Clase personalizada `PyUriTwc` (`nn.Module`) compatible con *Autograd* (PyTorch)
- Interaccion con el entorno modularizada
- Estructura vectorizada para:
  - Conexiones y pesos
  - Parámetros
  - Dinámica neuronal
- Modos de ejecución:
  - `forward`: interacción (estado persistente)
  - `forward_bptt`: secuencias temporales, utilizado para entrenamiento.

Implementación del Agente: PyUriTwc

Topología: conexiones y pesos representados mediante una unica matriz 11×11 junto a mascaras binarias para las conexiones sinapticas

- `mask_ex`
- `mask_in`
- `mask_gj`

Implementación del Agente: PyUriTwc

Topología: conexiones y pesos representados mediante una unica matriz 11×11 junto a mascaras binarias para las conexiones sinapticas

- `mask_ex`
- `mask_in`
- `mask_gj`

Dinámica Neuronal:

- Calculo de (E_{t+1}, O_{t+1}) en simultaneo
- $O_n = ReLU(S_n - Th_n)$
- E_n reemplazado por operaciones con enmascaramiento. (`torch.where`)

$$\begin{aligned} E_{new} = & M_{fire} \cdot Val_{fire} + \\ & M_{decay} \cdot Val_{decay} + \\ & M_{accum} \cdot Val_{accum} \end{aligned}$$

Motivación

Marco Teorico

Agente Bioinspirado e implementacion

Definición del Problema

Algoritmo de Entrenamiento: TD3

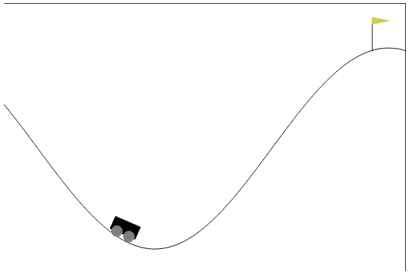
Dificultades y Estrategias de Mejora

Resultados

Conclusiones

El entorno Mountain Car Continous (MCC)

- **Objetivo**: alcanzar la cima
- **Dificultad**: el agente debe aprender a balancearse
- **Espacio de Estados**:
  - Posición: $p \in [-1.2, 0.6]$
  - Velocidad: $v \in [-0.07, 0.07]$
- **Espacio de Acciones**:
  - Fuerza de empuje: $f \in [-1, 1]$
- **Actualización**:
  - $\textit{velocidad}_{t+1} = \textit{velocidad}_t + \textit{fuerza} \cdot 0.0015 - 0.0025 \cdot \cos(3 \cdot \textit{posicion}_t)$.
  - $\textit{posicion}_{t+1} = \textit{posicion}_t + \textit{velocidad}_{t+1}$



El entorno Mountain Car Continous (MCC)

Funcion de recompensa

- **Bonificación por Éxito:** llegar a meta ($p \geq 0.45$)

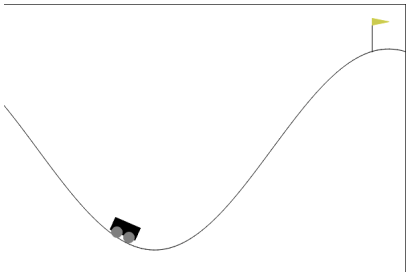
$$r_{t+1} = 100$$

.

- **Penalización por Esfuerzo:** En cada paso donde no se alcanza la meta

$$r_t = -0.1 \cdot a_t^2$$

.



Motivación

Marco Teorico

Agente Bioinspirado e implementacion

Definición del Problema

Algoritmo de Entrenamiento: TD3

Dificultades y Estrategias de Mejora

Resultados

Conclusiones

Twin Delayed Deep Deterministic Policy Gradient

- Metodo actor-critico
- Politicas en espacios de accion continuos
- Mejora a su predecesor **DDPG** en:
  - Sensibilidad a hiperparametros
  - Sobreestimacion de funcion Q.

Introduce tres estrategias:

- **Clipped Double-Q Learning**: usa dos criticos
- **"Delayed" Policy Updates**: la politica se actualiza con menor frecuencia que la funcion Q
- **Target Policy Smoothing**: agrega ruido a las acciones las *target networks*

Componentes del Algoritmo

- **Sequence Buffer** ($\mathcal{D}$): almacenamiento de transiciones alguna vez vistas durante el entrenamiento.
- **Target networks**: versiones rezagadas de los criticos y actor. Buscan estabilizar el entrenamiento.
- **Critico**: aproxima funcion de valor-accion $Q(s, a) \sim Q_\phi$

$$L(\phi, \mathcal{D}) = \mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}} \left[\left(Q_\phi(s, a) - (r + \gamma(1 - d) \max_{a'} Q_\phi(s', a')) \right)^2 \right] \tag{1}$$

- **Actor**: representa la politica del agente $\pi \sim \mu_\theta$

$$L(\theta, \mathcal{D}) = -\mathrm{E}_{s\sim\mathcal{D}} \left[Q_\phi(s, \mu_\theta(s)) \right] \tag{2}$$

TD3+BPTT: Recoleccion de Experiencia

1: Inicializar redes críticas Q_{ϕ_1}, Q_{ϕ_2} y actor recurrente μ_θ.
2: Inicializar *target networks* $\phi'_1 \leftarrow \phi_1, \phi'_2 \leftarrow \phi_2, \theta' \leftarrow \theta$.
3: Inicializar Sequence Buffer $\mathcal{D}$.
4: Definir longitud de secuencia de entrenamiento L y longitud de *burn-in* (opcional).
5: **for** cada paso $t = 1$ hasta T_{ep} **do**
6: Obtener acción y nuevo estado oculto: $a_t = \mu_\theta(o_t)$.
7: $a_t = \text{clip}(a_t + \epsilon, a_{min}, a_{max}), \quad \epsilon \sim \mathcal{N}(0, \sigma)$. *Aplicar Exploración*
8: Ejecutar a_t, observar recompensa r_t, nueva obs o_{t+1} y bandera d.
9: Almacenar transición $(o_t, a_t, r_t, o_{t+1}, d)$ en el episodio actual en $\mathcal{D}$.
10: **Paso de Actualización**
11: **end for**

TD3+BPTT: Paso de actualización

1: Muestrear secuencias de longitud L: $\tau = \{(o_k, a_k, r_k, o_{k+1}, done)\}_{k=1}^{L} \sim \mathcal{D}$
2: **Paso de Crítico:**
3: $\tilde{a}_k = \mu_{\theta'}(o_{k+1}) + \text{clip}(\epsilon, -c, c)$ *Target Policy Smoothing*
4: $y_k = r_k + \gamma(1 - d_k) \min_{i=1,2} Q_{\phi_i'}(o_{k+1}, \tilde{a}_k)$ *Clipped Double Q-Learning*
5: Actualizar ϕ_i vía BPTT: $\nabla_{\phi_i} \frac{1}{L} \sum_{k=1}^{L} (Q_{\phi_i}(o_k, a_k) - y_k)^2$
6: **if** Momento de actualizar Politica **then** *Delayed Policy Update*
7: **Paso de Actor:**
8: Actualizar θ vía BPTT: $\nabla_\theta \left(-\frac{1}{L} \sum_{k=1}^{L} Q_{\phi_1}(o_k, \mu_\theta(o_k)) \right)$
9: **Actualización Target Networs:**
10: $\phi_i' \leftarrow \tau\phi_i + (1 - \tau)\phi_i', \quad \theta' \leftarrow \tau\theta + (1 - \tau)\theta'$
11: **end if**

Motivación

Marco Teorico

Agente Bioinspirado e implementacion

Definición del Problema

Algoritmo de Entrenamiento: TD3

Dificultades y Estrategias de Mejora

Resultados

Conclusiones

Problema de convergencia a 0

Observamos el problema de **Retroalimentación escasa y diferida**

- El exito es un evento escaso
- Utilizar el motor conlleva penalizaciones
- $\pi^* \rightarrow$minimiza el esfuerzo ($a_t \sim 0$)

Mitigado mediante:

- Fomentar exploracion
- Muestreo de transiciones ponderado
- *Reward Shaping*

Estrategias de Mejora

Exploración con ruido correlacionado: Ornstein-Uhlenbekc(OU)

- Exitoso en **DDPG**
- Proceso estocastico con relacion temporal, el ruido en tiempo t depende del valor en $t-1$
- Permite exploraciones ruidosas manteniendo una misma dirección
- Prametros:
  - Θ:velocidad de reversión
  - μ: media a largo plazo
  - σ: volatilidad del ruido generado

Muestreo ponderado: Sequence Buffer

- Cada transicion individual tiene la misma probabilidad de ser elegida
- Asignacion de peso por episodio: $w_e = T_e - L + 1$
- Probabilidad de elegir un episodio: $p_e = \frac{w_e}{\sum_j w_j}$

Estrategias de Mejora

Reward Shaping

- Politicas que minimicen la cantidad de pasos
- Añadimos un termino adicional a la funcion de recompensa

$$\Phi(s) = 4 \cdot \text{clip}\left(\frac{x - x_{min}}{x_{max} - x_{min}},\ 0,\ 1\right)$$

- La recompensa modificada:

$$r'_t = r_t + \gamma\Phi(s_{t+1}) - \Phi(s_t)$$

Seleccion de mejor modelo e hiperparametros

Un modelo observado en el entrenamiento sera mejor cuando maximice:

- $\texttt{score} = \overline{R} - \max(0, (\overline{S} - 200) \cdot 0.1)$.

Estrategias de Mejora

Busqueda de mejores soluciones: Reward Shaping

- Politicas que minimicen la cantidad de pasos
- Añadimos un termino adicional a la funcion de recompensa

$$\Phi(s) = 4 \cdot \text{clip}\left(\frac{x - x_{min}}{x_{max} - x_{min}},\ 0,\ 1\right)$$

- La recompensa modificada:

$$r'_t = r_t + \gamma\Phi(s_{t+1}) - \Phi(s_t)$$

Motivación

Marco Teorico

Agente Bioinspirado e implementacion

Definición del Problema

Algoritmo de Entrenamiento: TD3

Dificultades y Estrategias de Mejora

Resultados

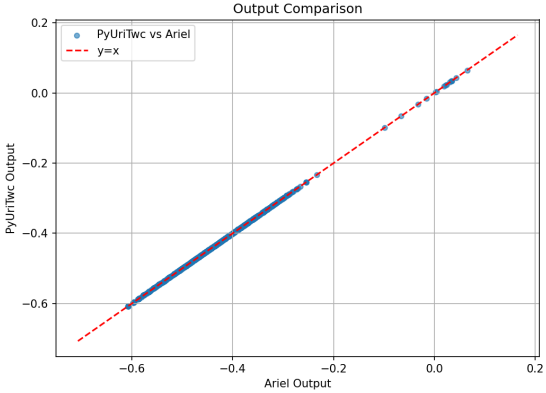
Conclusiones

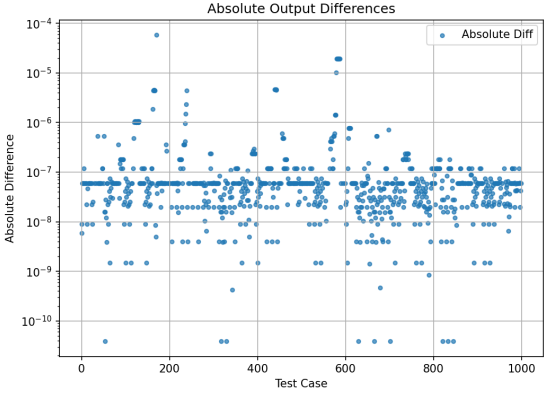
Recapitulacion de Objetivos

- Obtener una implementacion integrada en `PyTorch`
- Mantener fiabilidad numerica con el modelo de Ariel
- Aplicar algun metodo de entrenamiento estandar de DeepRl
- Resolver un problema de control continuo

Equivalencia Numerica

Ariel (Legacy) vs. PyUriTwc





Resultados de Referencia Publicados

Table: Resultados reportados en la literatura

Modelo	Avg Recomp.	Std Recomp.	Avg Pasos	Std Pasos
FIU+RS	98.43	0.15	–	–

Resultados Obtenidos

Table: Promedio sobre 100 episodios

Modelo	Avg Recomp.	Std Recomp.	Avg Pasos	Std Pasos	Avg Acción	Std Acción
Configuración 1						
Mejor	**94.13**	0.70	164.5	29.1	0.44	0.40
Final	94.04	0.64	158.6	30.9	0.45	0.42
Configuración 2						
Mejor	93.14	0.36	156.8	25.8	0.51	0.42
Final	92.21	2.20	162.4	44.0	0.55	0.43
Configuración 3						
Mejor	93.82	0.83	189.8	48.1	0.41	0.40
Final	92.84	1.24	175.3	38.2	0.48	0.42
Referencia	93.64	0.74	**124.5**	6.1	0.6	

Demo

Ver Demo

Motivación

Marco Teorico

Agente Bioinspirado e implementacion

Definición del Problema

Algoritmo de Entrenamiento: TD3

Dificultades y Estrategias de Mejora

Resultados

Conclusiones

Conclusiones

- . . .

¡Muchas Gracias!

